

# Transport Layer

## 11.1 INTRODUCTION

- The transport layer is the fourth layer from the bottom in the OSI reference model.
- Transport layer lies above the network layer in an end system.
- While the network layer and the other lower layers reside in all the end systems and intermediate systems (network nodes, routers), the transport layer is implemented only in the end systems. (see figure 11.1)

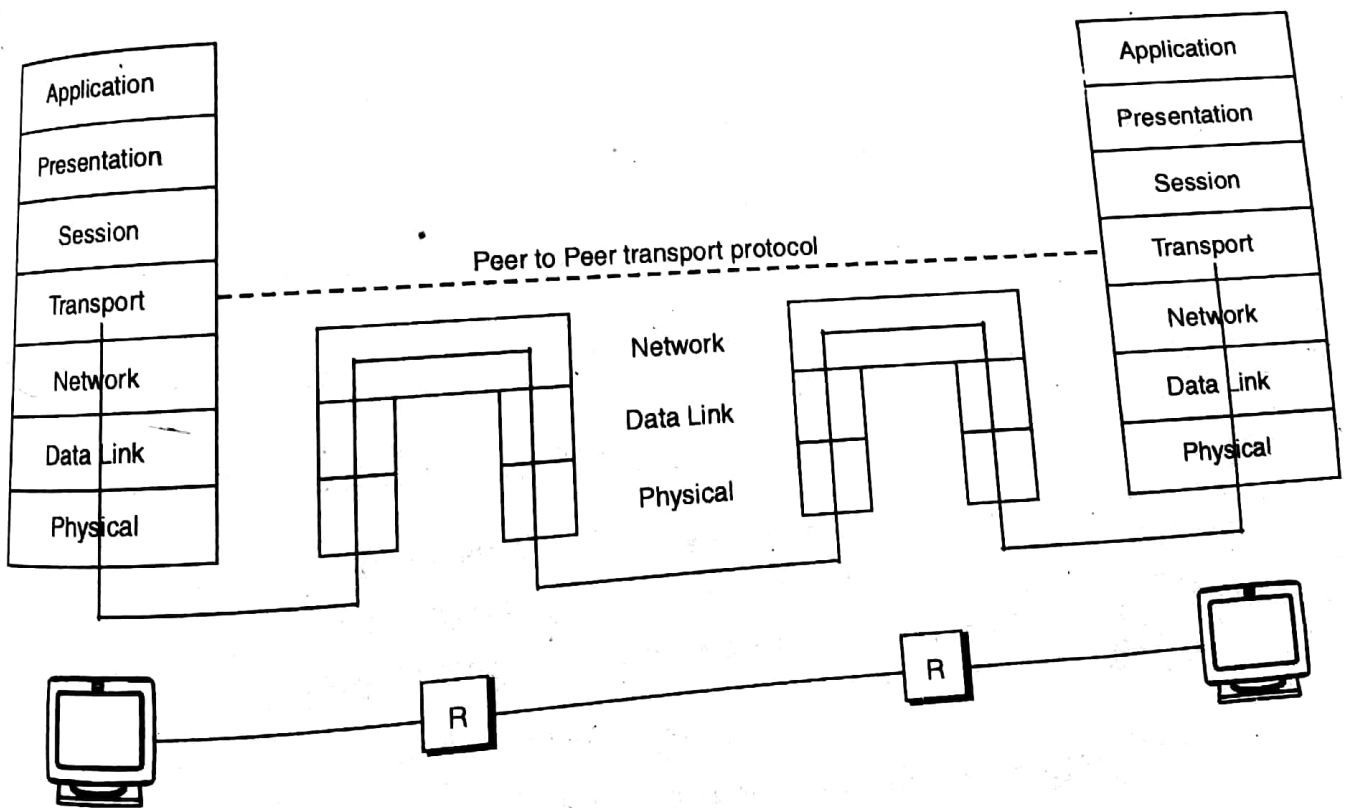
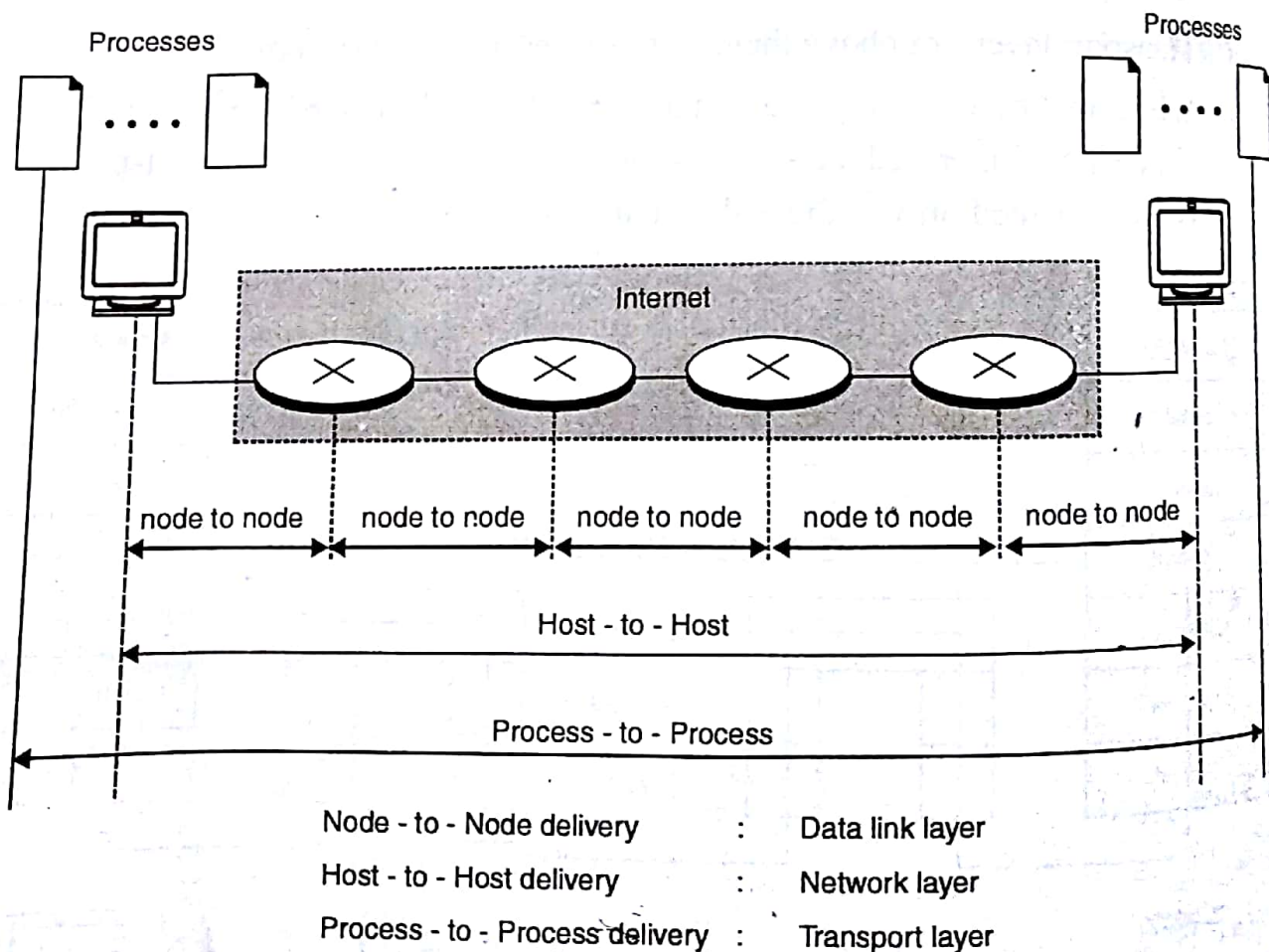


Fig. 11.1. Transport layer

- The interactions between peer transport layer entities are therefore, end-to-end and these are made possible by the data transfer service provided by the network layer.
- As discussed earlier, the data link layer is responsible for delivery of frames between two neighbouring nodes over a link. This is called node-to-node delivery.
- The network layer is responsible for delivery of datagrams between two hosts. This is called host-to-host delivery.
- The transport layer is responsible for delivery of entire message from one process running on first host to another process running on second host. This is known as process to process delivery.
- Thus, transport layer is responsible for process-to-process delivery of a message. The two processes communicate in a client/sever relationship.



**Fig. 11.2.** Process to process delivery by transport layer

## 11.2 TRANSPORT LAYER DESIGN ISSUES

11.3

The transport layer delivers the message from one process to another process running on two different hosts. Thus it has to perform number of functions or duties to ensure the accurate delivery of message. The various functions of transport layer are :

### 1. Establishing, maintaining and releasing transport connection :

- The transport layer establishes, maintains and releases end-to-end transport connection on the request of the upper layers.
- Establishing a connection involves allocation of buffers for storing user data, synchronizing the sequence numbers of data units etc.
- A transport connection is released at the request of upper layer.

### 2. Addressing

- In order to deliver the message from one process to another, an addressing scheme is required.
- Several processes may be running on a system at a time. In order to identify a correct process out of the various running processes, transport layer uses an addressing scheme called **port number**.
- Each communicating process has a specific port number.

### 3. Data Transfer

- The transport layer segments user data into smaller units and attaches a transport layer header to each unit forming a TPDU (Transport layer data unit).
- The TPDU is handed over to the network layer for its delivery to destination.
- The TPDU header contains port addresses of user processes, sequence number, acknowledgement number, checksum and other fields of various transport layer function.

### 4. Flow Control

- Like data link layer, transport layer also performs flow control.
- However, the flow control at transport layer is performed end-to-end rather than across a single link (node to node)
- Transport layer uses a sliding window protocol to perform flow control. Here, window size is dynamic and can vary in size depending on the receiver.

## 5. Error Control

- Transport layer also provides end-to-end error control facility.
- Data link layer can provide error control only between one node to another (see figure 11.3)
- Transport layer deals with several different types of errors :
  1. Content error due to corruption of bits (damaged data).
  2. Content error due to non-delivery of TPDUs (lost data).
  3. Content error due to duplicate delivery of TPDUs (duplicate data).
  4. Sequencing errors due to disordering of the TPDUs.
  5. Delivery of TPDUs to a wrong destination.
- Transport layer, has a built in error control checksum mechanism that detects and corrects errors by retransmission of transport data units.
- TPDUs carry sequence numbers so that missing TPDUs are detected. The missing TPDUs are retransmitted using an acknowledgement mechanism.
- Such sequence numbers of TPDUs also helps in putting the various data units in the right sequence and detecting duplicate TPDUs. These duplicate TPDUs can then be rejected.

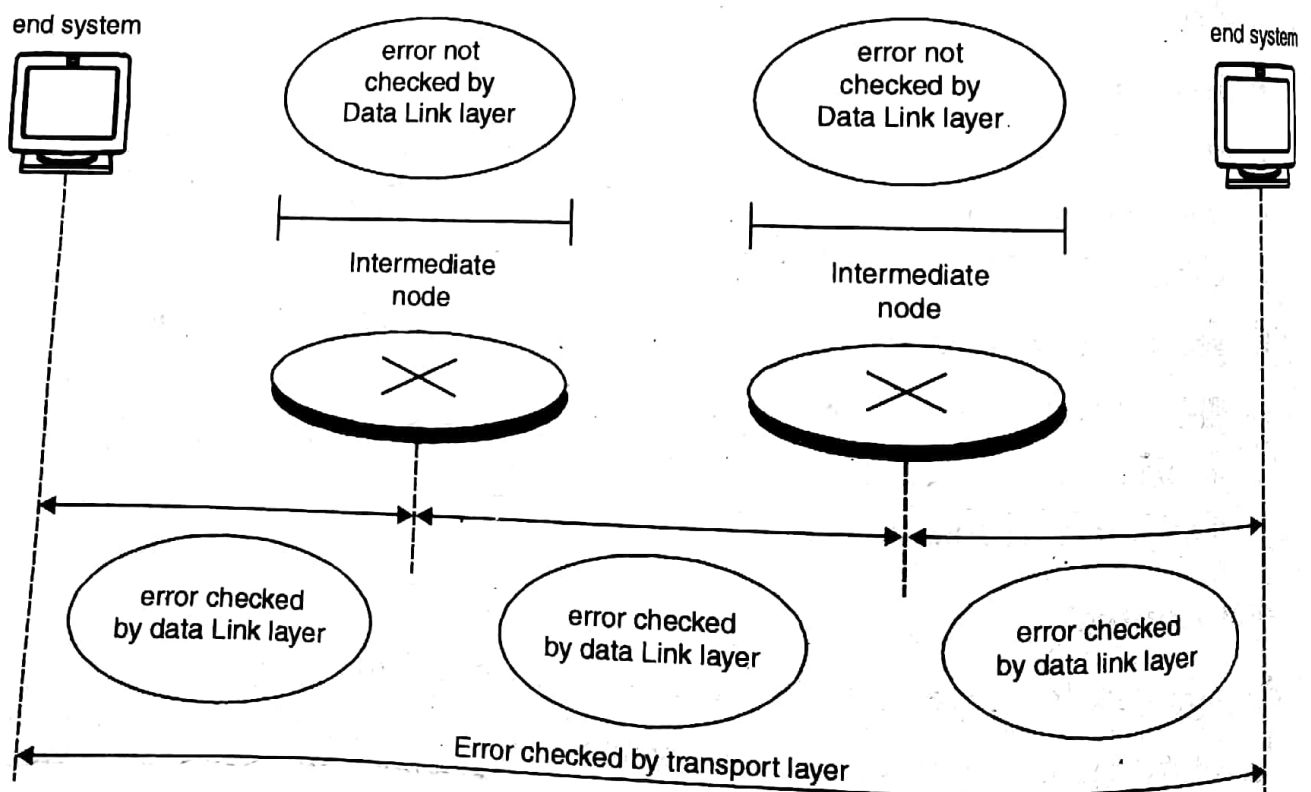


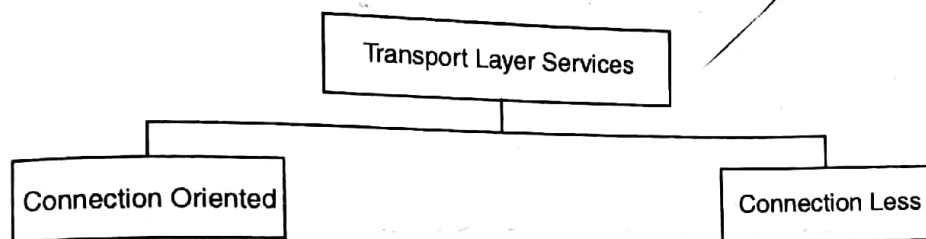
Fig. 11.3. Error Checking by Transport Layer.

### 11.3 Congestion Control

- The transport layer also handles congestion in the subnets that are using datagram service internally.
- Several different congestion control algorithms are used to avoid congestion.

## 11.3 CONNECTION ORIENTED AND CONNECTIONLESS SERVICES

A transport layer protocols can provide two types of services.



1. Connection Oriented Service
2. Connectionless Service

### 1. Connection Oriented Service

- In Connection oriented services, a connection is first established between a sender and the receiver.
- Then, transfer of the user data units takes place on this connection.
- At the end, the connection is released.
- During the establishment of the transport correction, the users can negotiate the quality of services desired.
- The transport layer protocols that provide connection oriented service are TCP and SCTP (Stream control transmission protocol).
- The connection oriented service is generally reliable.

### 2. Connectionless Service

- In this service the packets are sent from sender to receiver without the establishment or release of connection.
- In such a service, packers are not numbered.



- The packets may be lost, corrupted, delayed or disordered. Therefore connectionless service is unreliable.
- The transport layer protocol that provides this service is UDP.

## 11.4 ELEMENTS OF TRANSPORT PROTOCOLS

### 11.4.1 Addressing

- In order to deliver the data from one process to another we need an address.
- As discussed earlier, in order to deliver data from one node to another within a network, we require **MAC address** of source and destination. Such an addressing is implemented at data link layer and is called **physical addressing**.
- In order to deliver data from one network to another, network layer defined an addressing scheme called **logical addressing**. In logical addressing source and destination host need to know each other's **IP address**.
- Similarly, in order to deliver data from a process running on one machine to a process running on another machine, transport layer defines the **service point address or port numbers**.

### Port Numbers

- Each communicating process is assigned a specific port number (see fig. 11.4).
- In order to choose among multiple processes running on the destination host, a port number of destination process is needed.
- Thus, by using different port numbers, transport layer can simultaneously serve multiple processes.
- The port numbers are 16-bit integers between 0 and 65,535.
- In Internet model, the client process is identified by a port number that is chosen randomly by the transport layer software running on the client host.
- The server processes in internet model are not assigned port numbers randomly. If the processes at server are assigned port numbers randomly, the client process will not be able to know their port numbers. For this purpose, universal port numbers are used for servers. These are called well known port numbers.

- The port numbers are assigned by Internet Assigned Number Authority (IANA). IANA divides the port numbers in three different categories :
  - Well known ports :** The ports ranging from 0 to 1023.
  - Registered ports :** The port ranging from 1024 to 49,151. These are not assigned or controlled by IANA.
  - Dynamic ports :** The ports ranging from 49,152 to 65,535. These are neither controlled nor registered. They can be used by any process.
- Some examples of well known ports are 23–Telnet, 25–Simple Mail Transfer Protocol (SMTP), 80–Hypertext Transfer Protocol (HTTP).
- In client-server environments, a server process may need to maintain several simultaneous sessions with clients on different systems. Thus a single port supports multiple virtual connections.

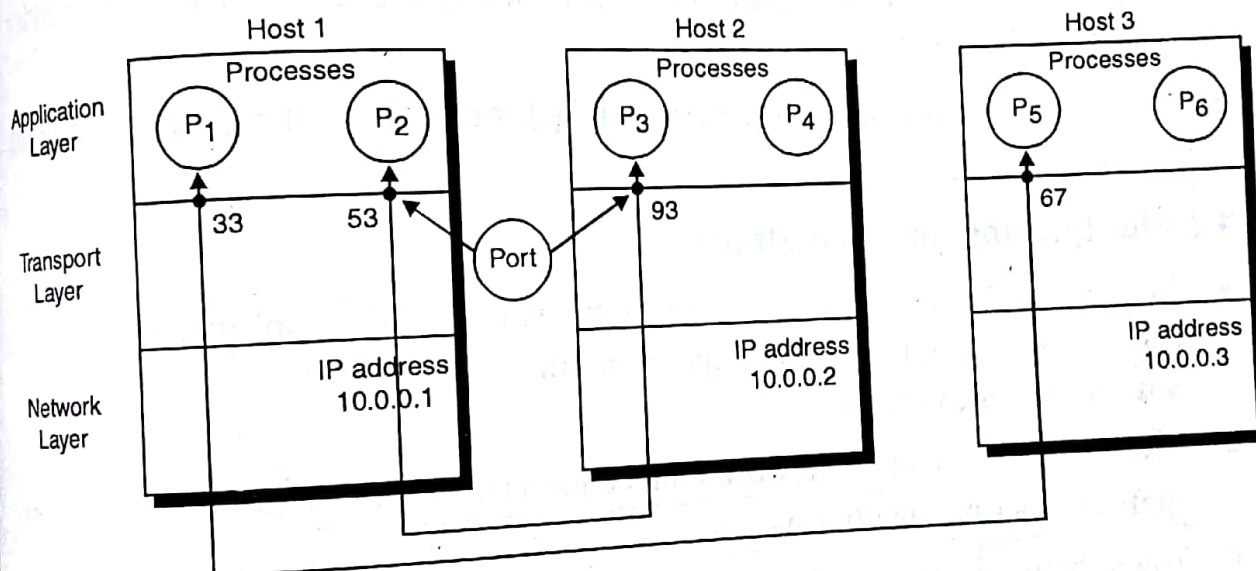
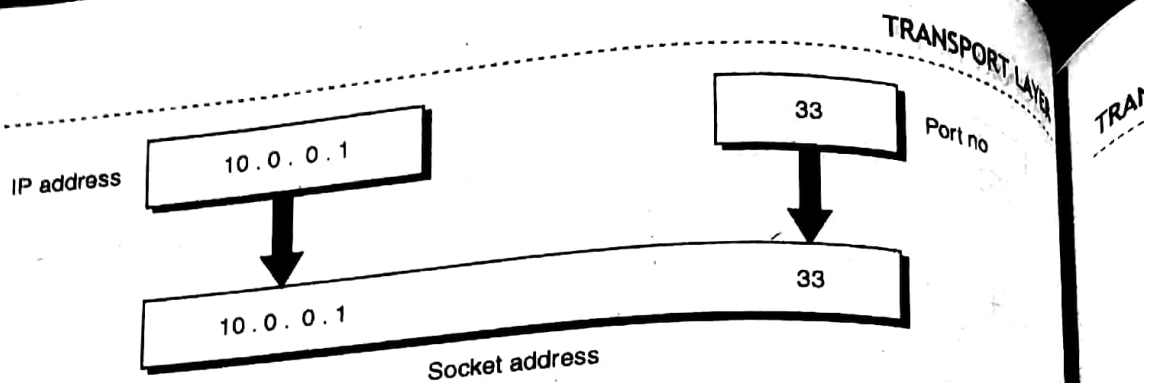


Fig. 11.4 TCP ports and connections

### Socket Address

- A socket address is a combination of IP address and port number (see fig. 11.5)
- In order to provide communication between two different processes on different host, we need both the IP address as well as port address i.e. socket address.
- The client socket address defines the client process uniquely and the server socket address defines the server process uniquely.



**Fig. 11.5 Socket Address**

- In order to provide process to process delivery, the transport layer protocol requires a pair of socket addresses. One for the client and other for the server.
- For example, as shown in figure 9.4. Process P1 of host 1 (having IP address 10.0.01 and port number 33) establishes a connection with process P5 on host 3 (having IP address 10.0.03 and port number 67).
- The socket address (IP address and port number) is usually a part of IP header and TPDU header.
- The IP header contains IP address and TCP or UDP header contains the port number.

1

#### 11.4.2 Multiplexing and Demultiplexing

- Modern operating systems support multitasking. In this, an operating system can run several applications at the same time. All these applications share the resources of the system.
- Like other resources, a network connection can also be shared by the various applications running on a system.
- When there are several running application that want to send packets and only one transport layer connection then transport layer protocols may perform multiplexing.
- The protocol accepts the messages from different processes having their respective port numbers and add headers to them.
- Then these packets are passed to network layer as shown in figure 11.6.
- The transport layer at the receiver end performs demultiplexing to separate the messages for different processes that are coming on a single connection.
- After checking for errors, the headers of messages are dropped and each message is handed over to the respective processes based on their port numbers.



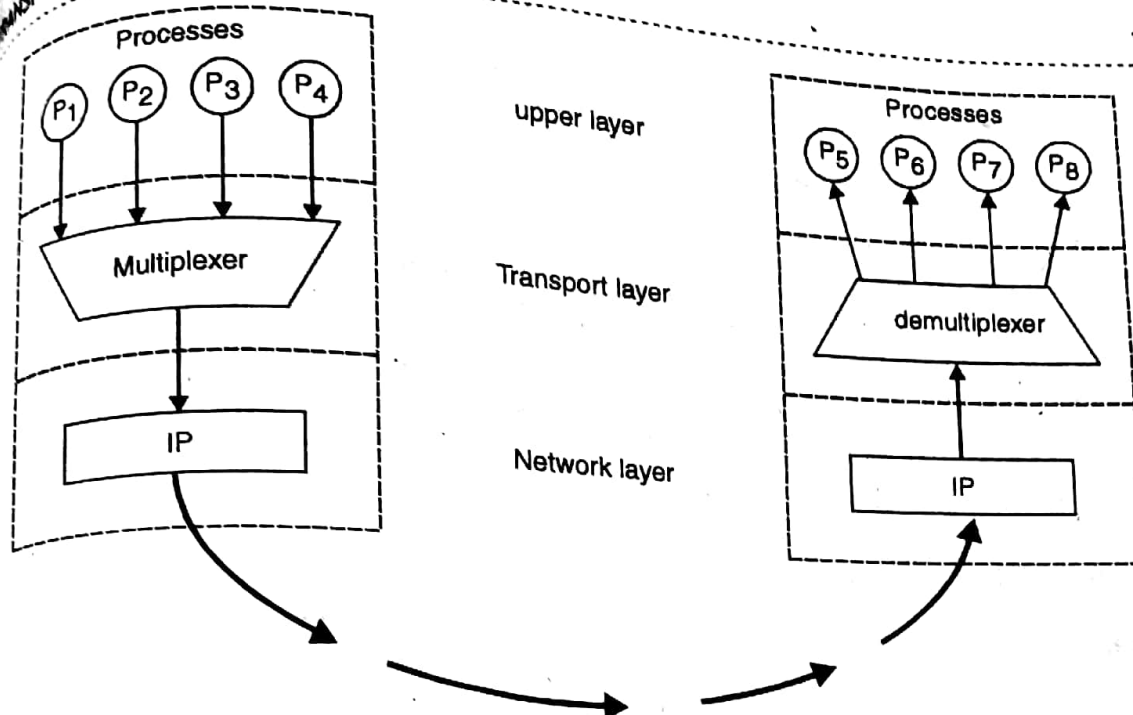


Fig. 11.6. Multiplexing and Demultiplexing.

### 11.4.3 Connection Establishment

- Before, either communicating device can send data to the other, the initiating device must first determine the availability of the other to exchange data and a pathway must be found through the network by which the data can be sent. This is known as **connection establishment**.

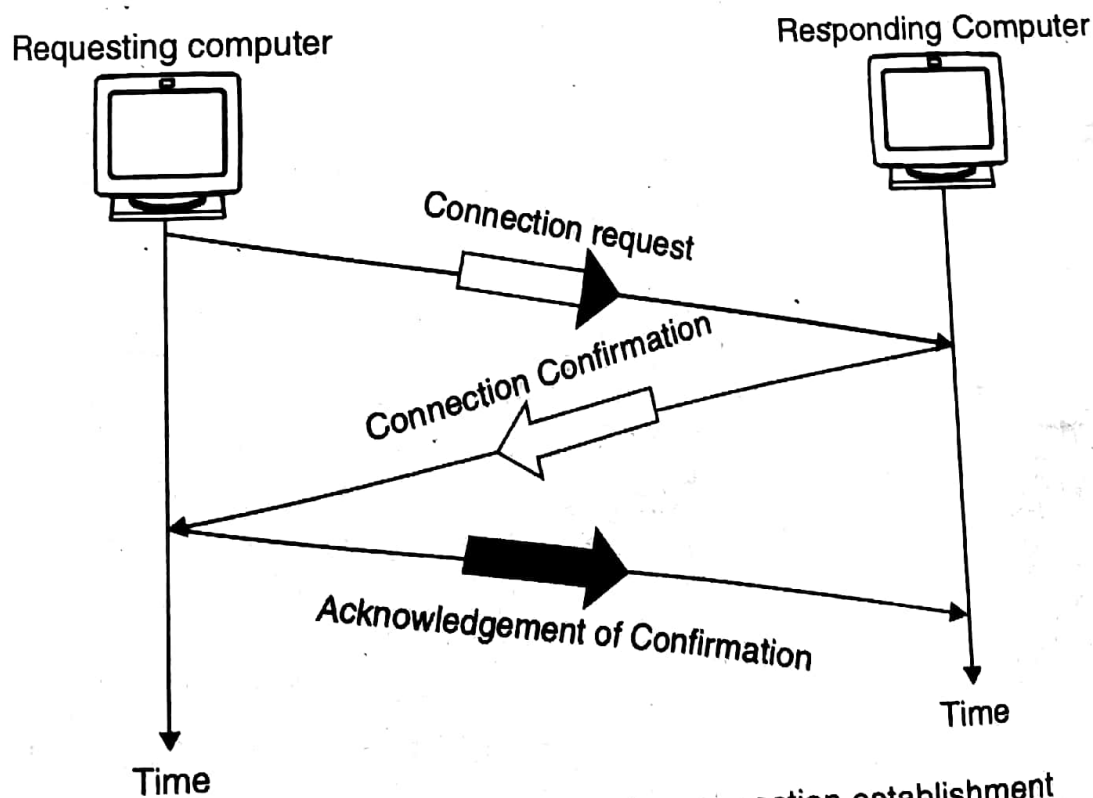
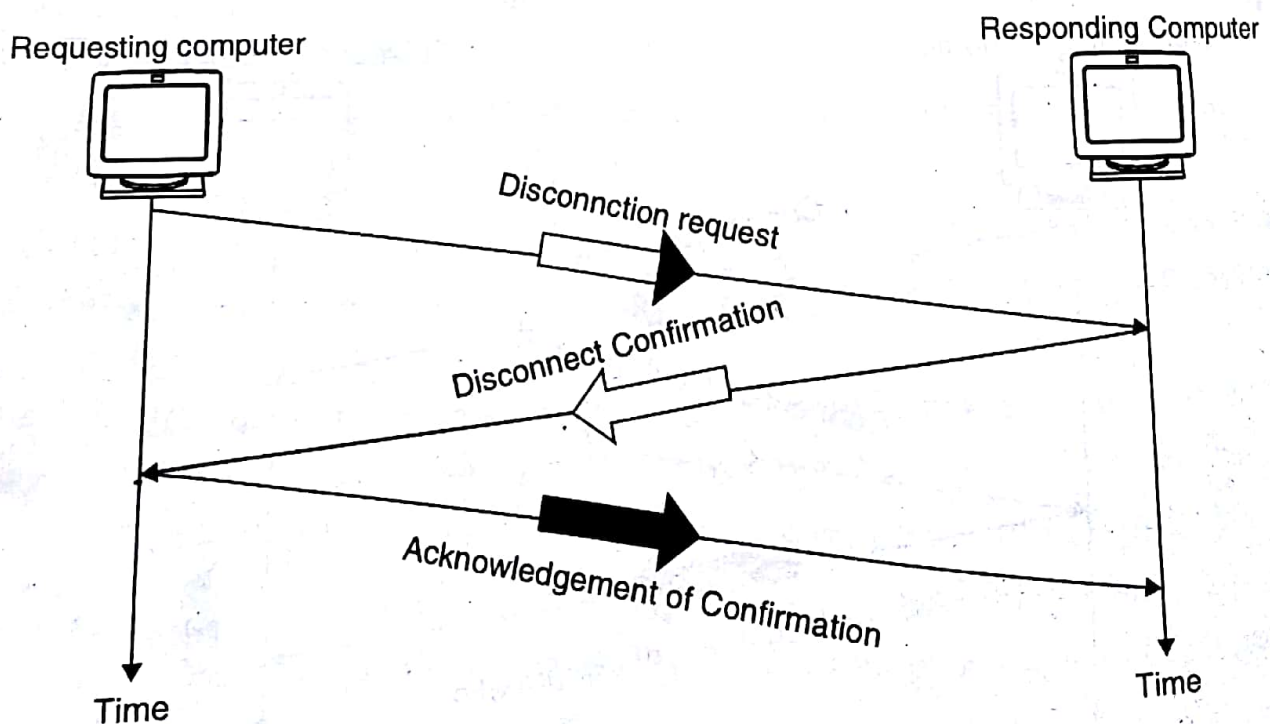


Fig. 11.7 Three way handshaking for connection establishment

- Connection establishment involves a **three way handshaking** mechanism (see fig. 11.7).
  1. The computer requesting the connection sends a **connection request** packet to the intended receiver.
  2. The responding computer returns a confirmation packet to the requesting computer.
  3. The requesting computer returns a packet acknowledging the confirmation.

#### 11.4.4 Connection Release

- Once all of the data have been transferred, the connection must be terminated or released.
- Connection termination also requires a three-way handshaking mechanism (see fig. 11.8)
  1. The requesting computer sends a disconnect request packet.
  2. The responding computer confirms the disconnect request.
  3. The requesting computer acknowledges the confirmation.



**Fig. 11.8** Three-way handshaking for connection termination.

## 11.5. TRANSPORT LAYER PROTOCOLS

As discussed earlier, the transport layer provides two types of services : connection oriented and connectionless.

For this, transport layer defines two different protocols Transmission Control Protocol (TCP) and User Datagram Protocol (UDP)

### 11.5.1 Transmission Control Protocol

- Transmission control protocol (TCP) is a connection oriented layer 4 protocol that provides reliable service between processes (applications) that reside in two different hosts (end systems).
- It uses the services of underlying IP layer which provides connectionless and unreliable service.
- TCP was formally defined in RFC 793. Later on, some changes were introduced to overcome errors and inconsistencies. These changes are documented in RFC 1122.
- The basic features of TCP are :

#### 1. It provides efficient method for numbering different bytes of data.

- TCP numbers all the data bytes transmitted in connection.
- The first bytes does not start with number 0. Rather it is numbered randomly between 0 to  $2^{32}-1$ . For example, if a random number is 1024 and the total number of bytes to be sent are 6,000, then the bytes are numbered from 1024 to 7023.
- After the bytes have been numbered, TCP assigns a **sequence number** to each segment that is being sent. The sequence number for each segment is the number of the first byte carried in that segment.  
For example, if 5000 bytes of data is to be transferred and the first byte is numbered 10,001 There are 5 segments of 1000 bytes each then the sequence number of first segment would be 10,001, sequence number of second segment would be 11,001 and so on.

#### 2. It provides stream data transfer

- With stream data transfer, TCP delivers an unstructured stream of bytes identified by sequence numbers.
- This facility is beneficial for the applications (processes) because the

processes need not to chop data into blocks before passing it to TCP. Instead TCP groups bytes into segments (a unit of data at transport layer is called segment) and pass them to IP for delivery.

### 3. It offers reliability

- TCP offers reliability by providing connection oriented, end to end reliable delivery of packets.
- It is able to achieve this by sequencing bytes with a forward acknowledgement number that indicates to destination the next byte of data the source expects to receive.
- Bytes that are not acknowledged within a specified time period are retransmitted.
- The reliability mechanism of TCP allows devices to deal with lost, delayed, duplicate or misread packets.
- A time-out mechanism allows devices to detect lost packets and request retransmission.

### 4. It provides efficient flow control

- TCP uses sliding window protocol to implement flow control.
- With this the receiving TCP process indicates the highest sequence number that it can receive without overflowing its internal buffers.

### 5. It provides full duplex operation.

- The mode of communication is two-way simultaneously.
- It means that TCP processes can both send and receive at the same time.

### 6. It provides multiplexing

- TCP allows several simultaneous upper layer conversations to be multiplexed over a single connection.

### 7. It provides connection oriented service

- The two TCP processes establish a connection between them, before any data is transferred.
- After the data is exchanged in both the directions the connection is terminated.